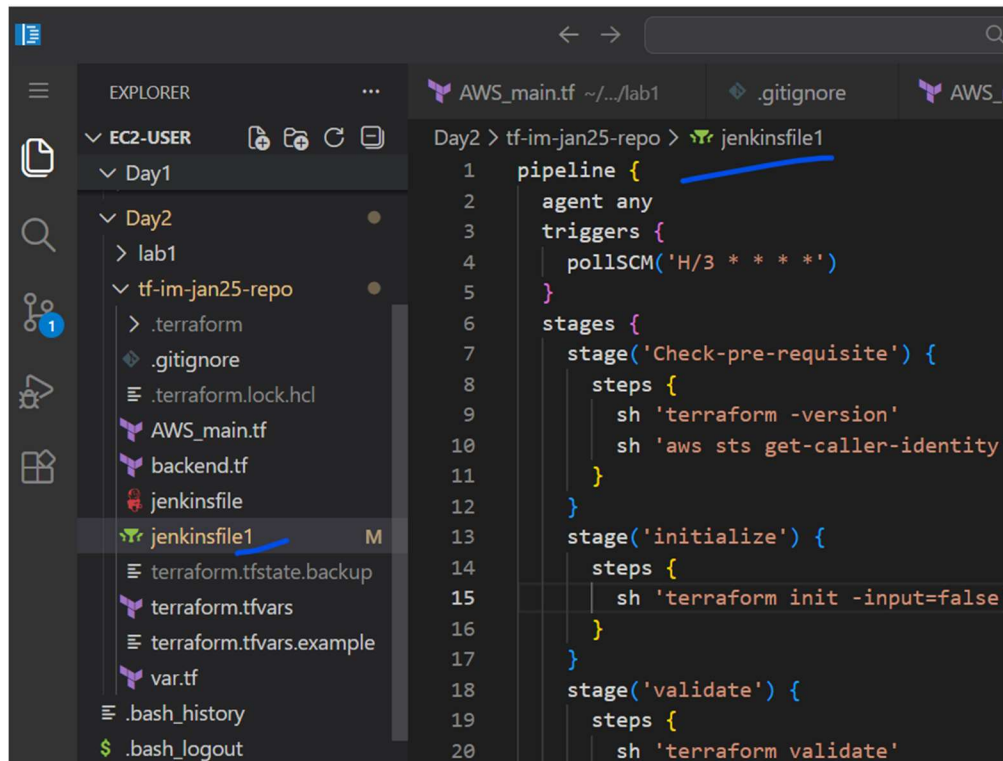


Steps

1. Create new jenkinsfile
Named: jenkinsfile1 in the root folder



2. Copy the content to jenkinsfile1

```
pipeline {
  agent any
  triggers {
    pollSCM('H/3 * * * *')
  }
  stages {
    stage('Check-pre-requisite') {
      steps {
        sh 'terraform -version'
        sh 'aws sts get-caller-identity'
      }
    }
    stage('initialize') {
      steps {
        sh 'terraform init -input=false'
      }
    }
    stage('validate') {
      steps {
        sh 'terraform validate'
```

```

stage('validate') {
  steps {
    sh 'terraform validate'
  }
}
stage('Format') {
  steps {
    sh 'terraform fmt -check'
  }
}
stage('plan') {
  steps {
    sh 'terraform plan -out=tfplan'
    sh 'terraform show -json tfplan > tfplan.json'
  }
}
}
}

```

3. Create an another branch

git checkout -b b2

```

1316814..1ba0754 main -> main
[ec2-user@ip-172-31-18-53 tf-im-jan25-repo]$ git checkout -b b2
Switched to a new branch 'b2'

```

4. Git commands

git add .

git commit -m "message"

```

[ec2-user@ip-172-31-18-53 tf-im-jan25-repo]$ git commit -m "added terraform stage"
[main 1ba0754] added terraform stage
 1 file changed, 34 insertions(+)
 create mode 100644 jenkinsfile1
[ec2-user@ip-172-31-18-53 tf-im-jan25-repo]$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 483 bytes | 483.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.

```

Git push

```
Ⓢ [ec2-user@ip-172-31-18-53 tf-im-jan25-repo]$ git push
fatal: The current branch b2 has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin b2

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.

● [ec2-user@ip-172-31-18-53 tf-im-jan25-repo]$ git push --set-upstream origin b2
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'b2' on GitHub by visiting:
```

5. Create a new job in Jenkins

Same configuration as job2 , but change the branch to “b2” and the Jenkins filename to the above one

The screenshot shows the Jenkins Configuration page for a job named 'vish-job3'. The left sidebar contains a 'Configure' section with a list of tabs: General, Triggers, Pipeline (which is highlighted), and Advanced. The main content area is divided into several sections: 'Branches to build' with a 'Branch Specifier (blank for 'any')' field containing the value '*/b2'; 'Repository browser' with a dropdown menu set to '(Auto)'; 'Additional Behaviours' with an 'Add' button; 'Script Path' with a text field containing 'jenkinsfile1'; and a 'Lightweight checkout' checkbox which is checked. A 'Pipeline Syntax' link is visible at the bottom of the configuration area.

Build the job manually for the first time



Status

Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Next Build



Console Output

```
Started by user Delvex labs
Obtained jenkinsfile1 from git git@github.com:Vishwanathms/tf-im-jan25-repo.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on node2 in /home/ec2-user/workspace/vish-job3
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential vish-github-cred
Cloning the remote Git repository
Cloning repository git@github.com:Vishwanathms/tf-im-jan25-repo.git
> git init /home/ec2-user/workspace/vish-job3 # timeout=10
Fetching upstream changes from git@github.com:Vishwanathms/tf-im-jan25-repo.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_SSH to set credentials vish-github-cred
> git fetch --tags --force --progress -- git@github.com:Vishwanathms/tf-im-jan25
Avoid second fetch
Checking out Revision 1ba075423c992abb0fc42b9ba13033de7d71e599 (refs/remotes/orig
Commit message: "added terraform stage"
First time build. Skipping changelog.
[Pipeline] }
```



Jenkins

/ vish-job3

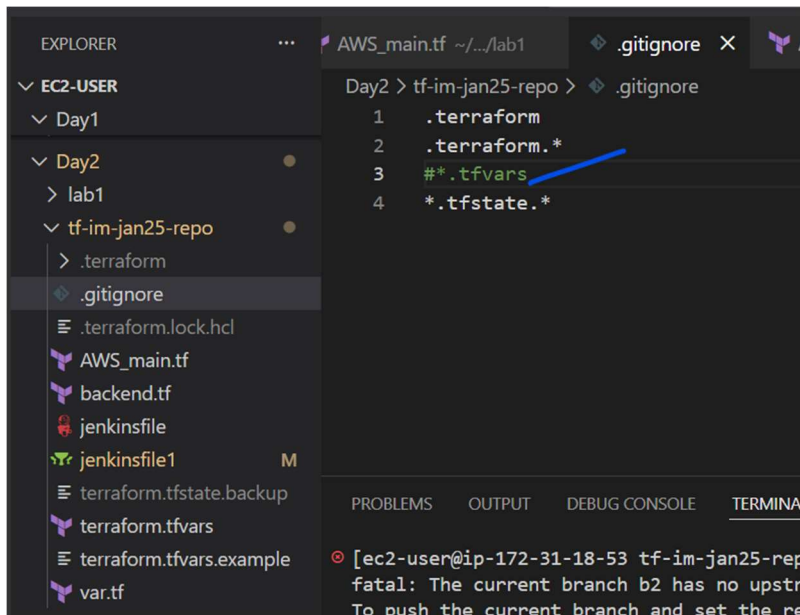
/ #1

/ Console Output

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (validate)
[Pipeline] sh
+ terraform validate
[32m[1mSuccess! The configuration is valid.
[0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Format)
[Pipeline] sh
+ terraform fmt -check
AWS_main.tf
backend.tf
var.tf
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (plan)
Stage "plan" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
```

It failed silently on the terraform fmt

6. To solve the problem



Comment the *.tfvars in the gitingore.
As the "Terraform fmt -check" needs it.

```
• [ec2-user@ip-172-31-18-53 tf-im-jan25-repo]$ terraform fmt -check
• [ec2-user@ip-172-31-18-53 tf-im-jan25-repo]$ git add .
• [ec2-user@ip-172-31-18-53 tf-im-jan25-repo]$ git commit -m "added the tfvars"
[b2 e4de1c8] added the tfvars
5 files changed, 31 insertions(+), 24 deletions(-)
create mode 100644 terraform.tfvars
• [ec2-user@ip-172-31-18-53 tf-im-jan25-repo]$ git push
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 938 bytes | 938.00 KiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pushed 7 (from 0)
```

Output



Jenkins

/ vish-job3

/ #2

/ Console Output

Status

Changes

Console Output

Edit Build Information

Delete build '#2'

Polling Log

Timings

Git Build Data

Pipeline Overview

Restart from Stage



Console Output

Started by an SCM change

Obtained `jenkinsfile1` from git `git@github.com:Vishwanathms/tf-im-jan25-repo.git`

[Pipeline] Start of Pipeline

[Pipeline] node

Running on **Jenkins** in `/var/lib/jenkins/workspace/vish-job3`

[Pipeline] {

[Pipeline] stage

[Pipeline] { (Declarative: Checkout SCM)

[Pipeline] checkout

Selected Git installation does not exist. Using Default

The recommended git tool is: NONE

using credential vish-github-cred

Cloning the remote Git repository

Cloning repository `git@github.com:Vishwanathms/tf-im-jan25-repo.git`

The job triggered automatically

**Jenkins**

/ vish-job3

/ #2

/ Console Output

```
[Pipeline] sh
+ terraform validate
[32m[1mSuccess! The configuration is valid.
[0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Format)
[Pipeline] sh
+ terraform fmt -check
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (plan)
[Pipeline] sh
+ terraform plan -out=tfplan
[0m[1maws_instance.main[0]. Refreshing state... [i

[1m[36mNote: Objects have changed outside c
[0m
Terraform detected the following changes made outside
last "terraform apply" which may have affected this p

[1m # aws_instance.main[0] has changed
[0m [33m~[0m[0m resource "aws_instance" "main" {
      id                                     = "i-0c5
[31m-[0m[0m public_ip
      tags                                  = {
```